AN ASYNC-FIRST KERNEL WITH SHARD-PER-CORE

2026-07-14

Contents

1 Introduction

1.1 The thesis

Hardware is asynchronous; the operating system and the userspace runtime should be too. Two projects, started independently from opposite ends of the software stack, built toward the same shape and converged on the same primitive: a waitable completion handle.

1.1.0.0.1 CharlotteOS

starts in the kernel and asks: *hardware is asynchronous with respect to the CPU, so why is the OS synchronous?* Its answer is to make the kernel async-first — cheap threads, an observer/waker event model, and system calls that are asynchronous by default, returning a completion capability that userspace can wait on.

1.1.0.0.2 sitas

starts in userspace and asks: *what does shared-nothing, shard-per-core concurrency look like when it is built out of Rust ownership instead of locks?* Its answer is a discipline: each shard owns its state, only the owning shard mutates it, and everything that crosses a shard boundary is an owned value moved through an explicit typed message.

Both projects independently converged. sitas's `Reply<T>` and `Notify` (awaitable completions) are the same object as CharlotteOS's `Observer` and `Waker`, just with different names. sitas's shard mailbox (`ShardSender`/`ShardReceiver`) is a bounded per-core typed queue — as are CharlotteOS's bounded per-LP `IPI_CMD_QUEUES`. sitas's `ShardLocal<T>` (lock-free, owner-checked, closure-based access) is a Rust-ownership alternative to CharlotteOS's `PerLp<T>` (lock-wrapped).

This document describes what happens when these two projects meet: a userspace runtime whose async model **agrees** with the kernel it runs on, instead of emulating async-first on a blocking foundation. That agreement removes the "retrofit tax" — the async-signal-safety, thread pools, and `io_uring`-over-blocking emulation that exist only because the runtime and the OS disagree about what is fundamental.

1.2 What was built and verified

The collaboration produced a booting, self-testing async-first kernel on both AArch64 and x86-64, with a full async syscall ABI (completion capabilities, CQ ring, exit-observer completion), a sitas-derived shard-per-core programming model in the kernel (`ShardLocal<T>`, `ShardMailbox<M>`, bounded IPI queues), and a sitas shard-per-core executor running as a user binary on CharlotteOS. Everything was verified on QEMU.

Key deliverables:

- **Completion-capability subsystem** — per-address-space capability tables, submit / complete / poll / wait / cancel / close, `submit_worker` (exit-observer), shared-memory CQ ring for zero-syscall completion delivery.
- **Syscall dispatch** — AArch64 SVC handler decodes `ESR_EL1.EC`, builds a `TrapFrame`, dispatches to a typed handler table. x86-64 SYSCALL trampoline (MSR-based entry, `swapgs` / `sysretq`).
- **Scheduler** — `RoundRobin` per LP, quantum timer, block / yield / wake, deferred thread reaper, per-LP thread pinning (`submit_to_lp` for sitas shard placement).
- **Cross-LP IPI** — bounded per-LP queues with first-class backpressure, typed closure dispatch (`IpiRpc::Closure`).
- **Shard-per-core programming model** — `ShardLocal<T>` (lock-free, owner-checked), `ShardMailbox<M>` (typed bounded queue).
- **Real-EL0 user thread** — compiled Rust binary runs at EL0, calls SVC from userspace, round-trips through the kernel's dispatch and returns.
- **sitas integration** — sitas executor (no_std) + sitas-charlotte ReactorBackend + `ShardRuntime::spawn_shard` via SVC #7. The sitas shard executor runs on CharlotteOS.

1.3 How this document is organized

- **Chapter 2** — Architecture: Logical Processors, Observers, capability tables, the scheduler, IPIs.
- **Chapter 3** — Execution model: threads, blocking, context switching, timers.
- **Chapter 4** — The async syscall ABI: the five operations, exit-observer completions, the CQ ring, cancellation, backpressure.
- **Chapter 5** — Programming model: invariants, ShardLocal, ShardMailbox, guidance.
- **Chapter 6** — Development: build, QEMU boot, self-tests, CI.
- **Chapter 7** — What we learned: bugs found on hardware, root causes, fixes, and key insights.
- **Chapter 8** — Current status: what is verified, known limitations, repositories.

1.4 Who this is for

Developers familiar with Rust (ownership, `Send`, `Future`), concurrency primitives, and basic OS architecture. Every claim is based on what was built and verified on QEMU AArch64 and x86-64. Unimplemented ideas are flagged.

2 Architecture

2.1 The logical processor

A **Logical Processor** (LP) is one hardware core. The kernel shards its state per-LP using `PerLp<T>`: an array of `RwLock<T>`, one slot per LP, indexed by the LP's identity. The LP identity is stored in a register (`TPIDR_EL1` on AArch64, `TSC_AUX` on x86-64) and read with a single instruction — it is always safe to call from any context, including interrupt handlers.

LP placement is **not advisory**: an LP *is* a core. When a sitas shard is bound to an LP it is bound to a core by construction. No `sched_setaffinity` race, no cpuset surprises.

2.2 Per-LP state

2.2.0.0.1 PerLp<T>

A boxed slice of `RwLock<T>`, one per LP. The lock masks interrupts on acquire, preventing ISR self-deadlock. Used for data accessed from both thread context and interrupt context (timer queues, LAPIC registers, interrupt depth counters).

2.2.0.0.2 ShardLocal<T>

(newer, sitas-inspired.) A lock-free per-LP container that stores its slots behind `UnsafeCell<T>` rather than `RwLock<T>`. Access is through a closure (`with(fn)` or `try_with(fn)`) that asserts:

- The caller is on the owning LP (panics otherwise).
- The slot is not already borrowed by a re-entrant call (a per-LP `AtomicBool` borrow flag, RAII-guarded).

The closure ensures the `&mut T` never escapes the accessor. Use `PerLp<T>` when interrupt-safety or cross-LP access is needed; use `ShardLocal<T>` for single-LP, thread-only data (it has no spin-lock acquire cost).

2.3 The observer / waker model

CharlotteOS's event mechanism reduces to one pattern: a thread registers its `Waker` (an `Observer` wrapping its `ThreadId`) on an `Observable`. When the observable fires, it calls `Observer::notify()`, which submits the thread to the system scheduler as ready.

2.3.1 Observable

trait: `fn register_observer(&self, observer: Weak<dyn Observer>)`.

2.3.2 Observer

trait: `fn notify(self: Arc<Self>)` — called by an Observable when the event occurs. The kernel's `Waker` implements `Observer` by calling `submit_ready_thread(tid)`, queueing the thread on the least-loaded LP.

2.3.3 Wiring

`sleep(50ms)` creates a `TimerEvent` (Observable), calls `block_thread(tid, &event)` to register the thread's `Waker` on the event and mark the thread Blocked, pushes the event into the per-LP timer queue, and yields. When the timer IRQ fires it processes the event, calls `signal()` which wakes the thread. The completion-capability model uses the exact same mechanism: a `Completion` is Observable, and `wait(cap)` blocks on it.

2.4 The scheduler

The system scheduler (`SYSTEM_SCHEDULER`) is a per-LP collection of pluggable `LpScheduler` instances. The only current implementation is `RoundRobin`: threads are queued in a FIFO, and a per-LP quantum timer (10 ms) marks a context-switch-pending flag.

Key operations:

- `spawn_thread(asid, entry)` — kernel-internal constructor that creates a Thread, adds it to the master table, and submits it to the scheduler. Returns a `ThreadId`. Userspace thread-spawn syscalls do not supply `asid`; the dispatcher derives it from the calling trap frame.
- `block_thread(tid, &event)` — registers the thread's `Waker` on the observable, marks the thread Blocked, removes it from the run queue. The thread saves its context on the subsequent `yield_lp()` and resumes from the same point when woken.
- `submit_ready_thread(tid)` — re-admits a thread to the least-loaded LP's run queue. Used by `Waker::notify()` and other wake paths.
- `yield_lp()` — sets the pending flag and calls `cond_yield_lp()`, which performs the register/stack switch if another thread is ready.

`abort()` removes a returning thread from the scheduler and moves it to `DEAD_THREADS` (a deferred reaper list). The reaper drops the thread from a different thread's context (after the context switch), so the thread's own kernel stack (which is freed on drop) is no longer in use.

2.5 Inter-processor interrupts (IPIs)

The `IPI_CMD_QUEUES` are per-LP bounded `ConcurrentQueue` queues (256 entries by default). Sending a cross-LP work item pushes an RPC to the target LP's queue and delivers an IPI. The target's IRQ dispatcher drains the queue in order.

`IpiRpc` carries kernel-internal RPCs (TLB shootdown, scheduler wakeup) and a `Closure(Box<dyn FnOnce() + Send>)` variant for arbitrary cross-LP work dispatch — the seed of sitas's typed `ShardMailbox<M>`.

Two push paths:

- `try_push_to` / `try_send_ipi_rpc` — returns `Err(rpc)` on full queue (first-class backpressure).
- `push_to` / `send_ipi_rpc` — non-fallible force-push for must-not-drop kernel RPCs. Evicts the oldest entry if full.

2.6 Capability tables

A per-address-space `IdTable<Arc<Completion>>` maps a small integer `CompletionCap` (an index into the table) to a kernel object naming an in-flight or completed operation. The table is bounded (submission backpressure: `submit` returns `WouldBlock` when full). `Completion` is `Observable`: a waiting thread's `Waker` is registered on it via `completion::wait(asid, cap)` inside the kernel. For real userspace syscalls, that `asid` is derived from the running thread's trap frame rather than supplied by userspace.

3 Execution model

This chapter describes how threads run, block, and yield — the runtime that the async-syscall ABI and the sitas reactor operate on top of.

3.1 Thread lifecycle

A thread is created inside the kernel with `spawn_thread(asid, entry_point)`. The `asid` determines whether it is a kernel thread (`asid == 0`, runs at EL1 / ring 0) or a user thread (`asid > 0`, drops to EL0 via `user_trampoline`). When userspace asks to spawn a thread, the syscall provides the entry point and placement; the caller's `asid` is taken from the trap frame.

3.1.1 Entry

The trampoline unmasks interrupts, calls the entry function, then calls `abort()` when the entry returns. `abort()` removes the thread from the scheduler and moves it to `DEAD_THREADS`; the reaper drops it (freeing its stack) from the next thread's context, after the context switch.

3.1.2 Blocking

A thread blocks by calling `block_thread(tid, &observable)` through a higher-level primitive (`sleep`, `wait`). The call marks the thread Blocked, registers its `Waker` as an observer on the event, removes it from the LP's run queue, and `yield_lp()` saves the thread's execution context and switches away. When the event fires, the waker submits the thread back to the run queue; the thread resumes exactly where it blocked.

3.1.3 Waking

`Waker::notify(tid)` calls `submit_ready_thread(tid)`, which adds the thread to the least-loaded LP's run queue and sends a wakeup IPI if that LP is idle and is not the calling LP. The thread runs at the next scheduling opportunity.

3.1.4 Yielding

`yield_lp()` sets the context-switch-pending flag and calls `cond_yield_lp()`. If the flag is set and another thread is runnable, the current thread's callee-saved registers and stack pointer are saved to its `ThreadContext`, and the next thread's are restored.

3.2 The quantum timer

Each LP's `RoundRobin` scheduler arms a periodic quantum `TimerEvent` (10 ms by default). When the timer fires, the observer sets the pending flag; the next `yield_lp()` (or the IRQ dispatcher's tail) performs

the context switch.

The quantum is re-armed after each context switch (or when the flag is cleared with nothing to switch to). An `armed` flag ensures exactly one quantum event is in flight at any time, preventing unbounded timer-queue growth from manual yields. The timer follows the same block-wake pattern as any other event: the hardware timer fires, the IRQ dispatcher processes the queue, `LpTimer::start()` re-arms for the next deadline.

3.3 The block-yield-wake cycle

This is the fundamental loop of the async runtime:

1. **Worker thread** calls `sleep(50ms)`. The timer event is created, the thread is blocked on it, the event is enqueued in the per-LP timer queue, the hardware timer is armed for the nearest deadline, and `yield_lp()` switches away.
2. **Other threads** run (or the LP idles). The timer IRQ fires at each quantum; the event-queue processes expired events.
3. **50 ms elapse.** The sleep event is at the front of the timer queue. `process_events` pops it, calls `signal()`, which upgrades the observer `Weak` and calls `Waker::notify()`, which calls `submit_ready_thread(tid)`.
4. **Worker** is now `Ready`. At the next scheduling point it runs, resuming immediately after its `yield_lp()` inside `sleep()`.

The block-wake cycle for `completion::wait` is identical: `complete()` signals the completion's observers (instead of a timer signal), waking the blocked thread.

3.4 Context saving

`switch_ctx(curr_sp_ptr, next_sp_ptr)` is the architecture-specific register-and-stack swap (AArch64: `x19–x30` and `TTBR0_EL1`; x86-64: callee-saved GPRs and `CR3`). It is called with both scheduler locks released, because it may permanently abandon the current stack (on the initial switch away from boot context). The post-switch code reaps dead threads.

4 The async syscall ABI

This chapter describes the userspace–kernel boundary for asynchronous system calls. It is the concrete interface that sitas (the shard-per-core userspace runtime) binds to, and it is the design surface where both projects converge. Every concept described here is implemented and verified on QEMU AArch64 and x86-64.

4.1 The five operations

4.1.1 submit — start an async operation

`submit(op_code, buffer) -> Result<CompletionCap, SubmitError>`. Returns immediately with a `CompletionCap` naming the in-flight operation. Ownership of the `buffer` (an owned `Vec<u8>`) transfers to the kernel for the operation's duration. Returns `SubmitError::WouldBlock` when the capability table is full (submission backpressure).

4.1.2 wait — the reactor's only sleep

`wait(cap) -> Result<(), CapError>`. Blocks the calling thread until the specified capability reaches terminal completion. Internally registers the thread's `Waker` as an observer of the `Completion` (which is `Observable`) and yields.

4.1.3 poll — non-blocking check

`poll(cap) -> Result<Option<Completed>, CapError>`. Returns `Ok(None)` while in flight; returns `Ok(Some(Completed { result, buffer }))` when terminal. The buffer ownership returns to the caller. Idempotent after completion (returns `None` once drained).

4.1.4 cancel — drop-as-cancellation

`cancel(cap) -> CancelState`. Requests cancellation of an in-flight operation. If the operation is still in flight, returns `CancelRequested`: the kernel retains any transferred buffer until it posts a terminal `Cancelled` completion (deferred reclaim, mirroring sitas's `defer_buffer_drop`). If the operation already completed, returns `AlreadyComplete`.

4.1.5 close — release a slot

`close(cap) -> Result<(), CapError>`. Frees a capability-table slot after its terminal completion was posted or its result was consumed by `poll`. Fails with `NotComplete` if the operation is still in flight.

The caller's address space is never a userspace parameter. On AArch64 the SVC path captures the running thread's address-space id in `TrapFrame.asid`; all address-space-scoped operations use that kernel-derived identity. Register `x0` is return-only for the typed wrappers; syscall arguments begin in `x1`. This is a security boundary, not a convenience convention: userspace must not be able to name another address space by passing an id.

4.2 The exit-observer completion path

The ABI's intended completion mechanism is declarative: **the worker thread exiting IS the completion event**. `submit_worker(worker_entry, result)` spawns a worker thread in the caller's address space and registers the returned capability as the worker's exit-observer. The worker performs its work and returns; the thread's `Thread::Drop` fires the exit-observer, which calls `complete()` and wakes any waiter. The worker never references the capability explicitly.

This is the exact design described in the kernel's own code (`scheduler/threads/mod.rs:63-69`) and is implemented in `crates/catten/src/completion/mod.rs` (`CompletionExitObserver`).

4.3 The CQ ring (completion queue)

A shared-memory io_uring-style ring for zero-syscall completion delivery. A single 4 KiB page contains a header (head, tail, capacity, overflow) and a circular array of entries (16 bytes each: `cap: u64, result: i64`).

- **Producer (kernel):** `write(cap, result)` advances the head. `complete()` writes to the ring alongside the observer signal.
- **Consumer (userspace):** `read()` advances the tail, returns `Option<CqEntry>`. Zero syscalls.
- **Overflow:** when the ring is full (head + 1 == tail modulo capacity), the write is dropped and the overflow counter is incremented (non-lossy detection).

The ring is allocated from a physical frame and mapped into the user address space via `PageType::UserData` (AP_EL0 accessible). The same physical memory is accessible from the kernel via HHDM. Currently demonstrated in kernel-side self-tests; userspace mapping is proven to work (the real-EL0 test maps a CQ ring alongside code and result pages).

4.4 Cancellation and the buffer-reclaim contract

When userspace drops a `CompletionCap` without awaiting it (or calls `cancel`):

1. The kernel marks the operation cancelling but **retains the buffer** — it may still be the target of DMA or kernel access.
2. A terminal `Cancelled` completion is eventually posted to the CQ ring, handing the buffer back.
3. On teardown (address-space exit), outstanding buffers are leaked rather than freed while the kernel may touch them (the kernel-side mirror of sitas's `mem::forget`-on-timeout rule).

This is not Unix-specific; it is an ownership contract. The ABI is specified against sitas's io_uring buffer-ownership discipline, and sitas's existing tests (`charlotte_abi.rs`) validate it.

4.5 Backpressure

- **Submission backpressure:** the per-AS capability table is bounded. `submit` returns `SubmitError::WouldBlock` when full. The caller must drain completions (freeing slots) before retrying.
- **Completion backpressure:** the CQ ring is bounded. When full, the kernel does **not** drop completions; it stops producing new ones for that shard and marks the queue overflow-pending.
- **Cross-LP backpressure:** `IPI_CMD_QUEUES` are bounded per target LP. `try_push_to` returns `Err(rpc)` on full queue. This matches sitas's bounded `ShardSender<M>` and the cache-coherence analysis in its architecture doc.

5 Programming model

This chapter describes the rules, patterns, and primitives for writing kernel code that respects the async-first, shard-per-core model. These invariants are derived from sitas and are enforced by the kernel's own concurrency design.

5.1 Invariants

5.1.1 Only the owning LP mutates its state

Per-LP data is accessed only from the LP that owns it. Cross-LP mutation must go through message dispatch (`ShardMailbox<M>`, IPI closure, or IPI RPC), never through a shared lock acquired from a foreign LP.

5.1.2 Messages are owned values

Everything that crosses a shard boundary is an owned value — a `Box<dyn FnOnce>`, an `IpiRpc`, or a typed `M` in a `ShardSender<M>`. No shared references, no `Arc<Mutex<...>` as the normal programming model. `Send + 'static` boundaries enforce this at compile time.

5.1.3 References to shard-local state must not escape

`ShardLocal<T>::with(fn)` provides `&mut T` inside a closure; the borrow is verified at runtime (owner-check + re-entrancy flag). The closure must not store the reference, send it to another thread, or cross an `.await`. The same rule applies to sitas's `ShardLocal<T>` userspace equivalent.

5.1.4 Work stays on its assigned LP

A thread spawned on LP N stays on LP N. To submit work to another LP, use `try_run_on_lp(target, closure)` or `ShardSender::try_send`. The kernel's IPI infrastructure and the sitas `ShardedSubmitter` are the two sides of this contract.

5.1.5 Observability returns owned snapshots

The kernel's house style is owned values for diagnostics. sitas's `RuntimeSnapshot` / `ShardSnapshot` pattern applies: never expose borrowed references into runtime internals. Self-tests and the demo illustrate this.

5.2 Primitives

5.2.1 ShardLocal<T>

Lock-free per-LP container. Use for single-LP, thread-only data.
Access: `sl.with(|val| { *val = 42; })`.

5.2.2 ShardMailbox<M>

Typed bounded per-LP queue. Cloneable `ShardSender<M>` (any LP can send), single-consumer `ShardReceiver<M>` (one per LP). `try_send(m)` returns `Err(m)` on backpressure.

5.2.3 IPI closure

`try_run_on_lp(target_lp, || { ... })`. Boxes arbitrary work and delivers it to the target LP's IPI queue. Returns the closure on full queue.

5.2.4 Completion API

`submit`, `submit_worker`, `complete`, `poll`, `wait`, `cancel`, `close`. The exit-observer path (`submit_worker`) is the intended default; the explicit `complete` is the escape hatch for non-thread-based work.

5.2.5 CQ ring

`CompletionQueueRing` for zero-syscall completion draining. Kernel side: `write(cap, result)`. Consumer side: `read() -> Option<CqEntry>`. Integration: `open_as_with_cq` attaches a ring to a capability table; `complete()` writes to it.

5.3 Rule-of-thumb guidance

- **Use** `PerLp<T>` for interrupt-accessed data and cross-LP access; **use** `ShardLocal<T>` for single-LP, thread-only data.
- **Use** `ShardMailbox<M>` (or `try_run_on_lp`) for cross-LP communication; do **not** lock an LP's state from another LP.
- **Use** `submit_worker` for fire-and-forget async kernel work; the worker just returns, and the exit-observer completes the capability.
- **Register** a CQ ring when creating an address space so that completions are visible to userspace without per-entry syscalls.
- **Accept** backpressure: bounded tables and queues return `WouldBlock` / `Err(m)`; retry or apply backpressure upstream. Never silently drop messages.

5.4 Working at the sitas–CharlotteOS boundary

The most useful mental model is that sitas is the userspace runtime and CharlotteOS is the substrate that makes sitas's assumptions native. sitas code should think in shards, owned messages, futures, and completion handles. CharlotteOS code should think in LPs, pinned threads, kernel-owned capability tables, and wakeable events. The two vocabularies describe the same shape at different privilege levels.

5.4.1 For userspace developers

Treat a CharlotteOS userspace program as a small `no_std` process with a `crt0` supplied by `catten-rt`. The program provides `cmain(args, input)` and uses `catten_rt::entry!(cmain)`; it does not define `_start`, an allocator, or a panic handler. The type `Input<N>` is the declaration of how many bytes the runtime should read before entering `cmain`. If the program does not need launch input, use `Input<0>`. Command-like values belong in `Args`.

Userspace application are interested in completion capabilities, mailbox capabilities, and returned handles.

When writing sitas-on-CharlotteOS code, keep the sitas model intact:

- A sitas shard is an LP-affine CharlotteOS user thread.
- `sitas-charlotte::CharlotteReactor::new(lp_id)` binds a reactor to the calling shard's LP.
- Async operations return kernel-owned completion capabilities. A completion capability is opaque; wait for it, poll it, close it, or pass it through a typed channel when the ABI permits that.
- Cross-shard work is an owned message. Prefer sitas channels and futures over shared mutable state.
- Backpressure is normal. Bounded queues and capability tables are part of the contract, not exceptional conditions to paper over.

The current `basic_kv` image is the reference smoke test. It enters via `crt0`, uses `Args([])` and `Input<0>`, constructs a Charlotte reactor for LP0, starts a one-shard `ShardedKv`, writes three keys, reads one back, and reports `total_len == 3` through the configured output page. If this stops working, either the userspace launch contract, the syscall ABI, the loader mapping, or the sitas runtime boundary has regressed.

5.4.2 For kernel developers

The kernel side must preserve the authority and ownership boundaries that make the userspace model sound:

- **Caller identity is kernel authority.** Syscalls derive the caller's address space from `TrapFrame.asid`; never accept ASID, page-table roots, or equivalent authority from userspace registers or config pages.
- **Handles are capabilities.** Completion and mailbox ids are scoped to the caller's address space. Validate them in the per-AS table before acting.
- **Completions must be non-lossy.** A CQ overflow counter is useful diagnostics, but a terminal completion must either be delivered, retained in a kernel backlog, or prevented by submission backpressure.
- **Buffers have one owner.** While an async operation owns a user buffer, userspace must not mutate or free it. Cancellation still ends in a terminal completion so ownership can return or be deliberately leaked during teardown.

- **Placement is real.** A sitas shard maps to an LP-pinned user thread. `SPAWN_THREAD` takes an entry virtual address and target LP; the address space comes from the caller.
- **Do not smuggle shared kernel state into userspace.** Shared pages such as the CQ ring are data queues with a narrow layout, not references to kernel objects.

The present sitas smoke loader now consumes a stripped AArch64 ELF image rather than a flattened binary. It maps each `PT_LOAD` segment at its linked virtual address, chooses `UserCode`, `UserRoData`, or `UserData` from the segment flags, rejects writable-executable segments, rejects LOAD segments that overlap within one 4 KiB page, and relies on the freshly zeroed page frames to satisfy the BSS contract. The `catten-user` linker script therefore page-separates executable text, read-only data, and writable data.

This is still a self-test loader, not yet a general userspace process loader: the config page, CQ page, input page, and heap are still mapped at fixed CharlotteOS addresses, and there is no dynamic argument/environment layout. Treat larger sitas binaries as both runtime tests and loader-contract tests.

6 Development

6.1 Build targets

The kernel builds for three architectures via custom target JSON specs in `target_specs/`:

- `aarch64-unknown-none-catten.json`
- `x86_64-unknown-none-catten.json`
- `riscv64gc-unknown-none-catten.json` (planned, not actively maintained)

Build command (headless, no display dependency):

```
cargo build --package catten \
  --target target_specs/aarch64-unknown-none-catten.json \
  --no-default-features --features acpi
```

The `display` feature requires a C library for the flanterm framebuffer terminal; headless boot uses serial output only (AArch64: PL011 UART; x86-64: 16550 COM1). AArch64 boots via `scripts/boot-smp1.sh` (single-core) or `scripts/boot-aarch64.sh`. x86-64 boots via `scripts/boot-x86.sh` (requires `-cpu max` for `RDTSCP`/`FSGSBASE`).

6.2 Self-tests

There is no host-side `cargo test`; all tests are **boot-time kernel self-tests** (∼whitebox integration tests) run from `self_test::run_self_tests()` in `main.rs:104`, called after `INIT_BARRIER.wait()`. Each test uses `assert!()` / `assert_eq!()` (panic = failure). Current test suite (both arches pass):

- `self_test/completion.rs` — buffer ownership, cancel / deferred-reclaim, backpressure
- `self_test/syscall.rs` — dispatch table routing via synthetic TrapFrame
- `self_test/ipi.rs` — bounded queue, backpressure, closure dispatch
- `self_test/shard.rs` — `ShardLocal<T>` owner-check / re-entrancy guard, `ShardMailbox<M>` send/recv
- `self_test/el0.rs` — real-EL0 user thread (AArch64 only)
- `self_test/cq.rs` — CQ ring write/drain/overflow
- `self_test/cq_completion.rs` — CQ ring integrated with completion subsystem

6.3 Async-syscall demo

`crates/catten/src/demo.rs` spawns a coordinator + worker thread from `bsp_main` (after the scheduler is active). The coordinator calls `submit_worker`, which spawns a worker and returns a capability that fires when the worker exits; the coordinator blocks on `wait(cap)`. The worker sleeps, returns,

and the exit-observer completes the cap, waking the coordinator. This exercises the full submit–async-work–complete–wake loop.

Output at boot:

```
Testing Complete. All Tests Passed!
[async-demo] coordinator: submitted worker, now awaiting completion...
[async-demo] worker: performing async work (sleep 50ms)
[async-demo] worker: work finished, exiting (completion fires on exit)
[async-demo] coordinator: COMPLETION RECEIVED, result Ok(42)
[async-demo] SUCCESS: async syscall round-trip complete (via thread-exit
             observer)
```

6.4 CI

GitHub Actions (`.github/workflows/ci.yml`) builds both architectures with `-D warnings` and runs an automated QEMU boot gate on the `dev` branch (grepping for "Testing Complete. All Tests Passed!").

6.5 Where to start reading

- `crates/catten/src/completion/mod.rs` — the completion subsystem (capability table, submit/complete/poll/wait/cancel/close, `submit_worker`, exit-observer).
- `crates/catten/src/completion/cq.rs` — the CQ ring buffer.
- `crates/catten/src/syscall/mod.rs` — AArch64 syscall dispatch table.
- `crates/catten/src/demo.rs` — the end-to-end async demo.
- `crates/catten/src/cpu/scheduler/` — threads, wakers, `RoundRobin`, `block_thread`.
- `crates/catten/src/cpu/multiprocessor/ipi.rs` — bounded IPI queues and the `IpiRpc` enum.
- `crates/catten/src/cpu/scheduler/threads/mod.rs:63-69` — the completion-capability design comment.

7 Current Status

7.1 Verified (boots on QEMU, zero panics)

- **AArch64:** boots, all self-tests pass, the async-syscall demo succeeds with `Ok(42)` via exit-observer, real-EL0 user thread executes at EL0 and dispatches syscalls. The sitas `basic_kv` user binary runs through `crt0/cmain`, spawns a shard via SVC #7, and reports `total_len == 3`.
- **x86-64:** boots with `-cpu max`, all self-tests pass, the async-syscall demo succeeds with `Ok(42)` via exit-observer. (x86-64 ring-3 SYSCALL userspace is implemented but not yet exercised with a user thread.)
- **Completion subsystem:** submit, complete, poll, wait, cancel, close, `submit_worker` (exit-observer), CQ ring, CQ ring integrated with AS, non-lossy CQ overflow backlog.
- **Syscall dispatch:** AArch64 SVC handler for completion operations, thread exit, LP-pinned spawn, CQ wait, and mailbox capability operations. The caller's address space is taken from `TrapFrame.asid`, never from a userspace ASID parameter. x86-64 SYSCALL trampoline exists, connected to the dispatch table.
- **IPI:** bounded per-LP queues, backpressure, typed closure dispatch, both architectures unstubbed. Cross-core SGI delivery works on QEMU AArch64 GICv3 with `-smp 2`; the boot gate exercises kernel `ShardMailbox` fan-out, EL0 cross-LP completion, and EL0 ping-pong over mailbox endpoint capabilities on both the default accelerator path and explicit TCG.
- **ShardLocal<T> / ShardMailbox<M>:** implemented and tested in-kernel.
- **Scheduler:** RoundRobin, quantum timer, block/yield/wake, deferred reaper, per-LP thread pinning (`submit_to_lp`).
- **sitas integration:** sitas-core (no_std executor) compiles for aarch64-unknown-none. sitas-charlotte (ReactorBackend + ShardRuntime::spawn_shard via SVC #7) links against the kernel's syscall ABI. The current user binary runs `basic_kv`, creates a `ShardedKv`, spawns a shard thread, and verifies the result from EL0.
- **CI:** GitHub Actions builds both architectures with `-D warnings`, QEMU boot gate.

7.2 Known limitations

- **x86-64 ring-3 userspace:** SYSCALL entry trampoline exists, GS base and IRETQ return path remain to be wired. The EL0 test and real-EL0 user thread are AArch64-only.
- **General userspace loader:** the sitas smoke now uses a segment-aware ELF loader for its embedded user image, including per-segment page permissions and zero-filled BSS. It is still a fixed-address self-test harness for config, CQ, input, and heap pages, not a general process loader with argv/env/auxv-style setup.

7.3 Repositories and branches

- **sitas** (`reactor-handle-seam`): `ReactorBackend` trait, ABI reference model (`charlotte_abi.rs`), `sitas-core` no_std crate (ringbuf, ShardRuntime, ShardLocal, Shard-

Mailbox, basic_kv), `sitas-charlotte` crate (ReactorBackend + ShardRuntime backend, with ASID kept out of the userspace API).

- **CharlotteOS** (`dev`): consolidated kernel — completion subsystem, syscall dispatch, bounded IPI, ShardLocal, ShardMailbox, CQ ring, real-EL0 test, exit-observer completion, SPAWN_THREAD syscall, sitas user binary, developer manual (31-page PDF).

7.4 Build and boot

AArch64:

```
cargo build --package catten \
  --target target_specs/aarch64-unknown-none-catten.json \
  --no-default-features --features acpi
./scripts/boot-smp1.sh
```

x86-64:

```
cargo build --package catten \
  --target target_specs/x86_64-unknown-none-catten.json \
  --no-default-features --features acpi
./scripts/boot-x86.sh
```

sitas user binary (requires nightly + build-std):

```
cargo +nightly build --manifest-path crates/catten-user/Cargo.toml \
  --target crates/catten-user/aarch64-unknown-none.json \
  -Zbuild-std=core,alloc,compiler_builtins
```

8 What We Learned

Bootstrapping an async-first kernel on real hardware (QEMU) surfaced a series of latent bugs that static analysis and self-tests never caught. Each fix enabled the next layer of work. This chapter records the most instructive, in the order they were discovered, and the design insights they produced.

8.1 Bugs found on hardware

8.1.1 Panic handler recursed through heap-dependent `logln`

Symptom: any fault during early boot (before the heap allocator was ready) silently triple-faulted instead of printing a panic message — making *every* early bug look like a mysterious hang.

Root cause: the panic handler used `logln!`, which routes through `INT_STATE` — a `LazyLock` whose first-use initialization allocates from the kernel heap (`alloc::vec!`). A fault before the heap was ready caused the panic handler's `logln!` to trigger a heap allocation → another fault → re-entered the panic handler → infinite recursion → stack overflow → triple fault.

Fix: the panic handler now uses `early_logln!` (direct serial writes, no heap, no `INT_STATE`). A panic handler must never depend on the allocator it might be reporting a failure about.

This was the single most valuable fix: it unmasked every subsequent fault as a visible panic instead of a silent triple-fault.

8.1.2 Blocked threads lost their execution context

Symptom: a thread that blocked in `wait()` (or `sleep()`) restarted from its entry point when woken, instead of resuming where it blocked. The async-syscall demo coordinator reset entirely on every wake.

Root cause: `block_thread` cleared the LP scheduler's `current_handle` for the running thread, so the subsequent `cond_yield_lp` read `curr_tid == None` and `switch_ctx(null, ...)` **skipped saving** the blocking thread's execution context.

Fix: a self-blocking thread now stays `current_handle` until the yield saves its context; `RoundRobin::next` declines to re-queue a `Blocked` thread. This was foundational: `wait()` and `sleep()` had never been exercised before.

8.1.3 RwLock held-across-call deadlocks in `sleep()` and `abort()`

Symptom: `sleep()` worked reliably with diagnostic logging but hung without it — a classic heisenbug.

Root cause: `sleep()` held a `SYSTEM_SCHEDULER.read()` guard (plus the LP scheduler lock) across `SYSTEM_SCHEDULER.write().block_thread(...)` — a read-then-write RwLock self-deadlock. The diagnostic version happened to bind the thread ID first, releasing the guard. `abort()` had the same pattern with the LP scheduler lock.

Fix: bind the value out of the scrutinee first so the temporary guards drop before the re-locking call. This applies to *any* use of RwLock where a read guard scope is extended by an `if let` or `match` binding.

8.1.4 Quantum timer grew the queue without bound

Symptom: after extended operation, the per-LP timer queue contained thousands of quantum events, consuming unbounded memory.

Root cause: `clear_ctx_switch_pending` enqueued a new quantum `TimerEvent` on every yield, not just on quantum expiry. Manual yields accumulated quantum events forever.

Fix: an `armed` flag keeps exactly one quantum event in flight (the quantum's own firing clears it). Also, `cond_yield_lp` re-arms even when there is nothing to switch to (previously the timer stopped when only one thread was runnable), and `process_events` stops the level-triggered timer when the queue drains.

8.1.5 A thread cannot free its own kernel stack

Symptom: a returning kernel thread hung during teardown.

Root cause: `ThreadContext::Drop` deallocates the kernel stack — but the returning thread is still executing on that stack when `abort` drops it. The CPU eventually faults on freed memory.

Fix: a **deferred reaper**. `abort` moves the dying thread to `DEAD_THREADS` (`IdTable::take_element` extracts it without dropping), and `cond_yield_lp` calls `reap_dead_threads()` *after* switching away, so the stack is freed from a different thread's context.

8.1.6 Stack allocator never registered guard pages

Symptom: `deallocate_stack` always failed with `InvalidStack`, causing the reaper to leak every freed stack.

Root cause: `allocate_stack` never populated `KERNEL_GUARD_PAGE_SET`, so `validate_stack` could never find the guards.

Fix: register lower and upper guard pages on allocation and rewrite `deallocate_stack` to derive the page count from them. Reference-count guard pages (`BTreeMap<VAddr, usize>`) so adjacent stacks sharing a guard survive until both are freed.

8.1.7 LA57 paging-mode mismatch on x86-64

Symptom: a `#GP` with `RAX = 0xff90000000000000` (non-canonical under 4-level paging) during kernel-heap large-page mapping.

Root cause: `get_vaddr_sig_bits()` read the CPU's *maximum* linear address width from CPUID. With `-cpu max` that is 57, so the kernel selected the 57-bit address map. But Limine boots with 4-level paging (48-bit), where addresses in the 57-bit map are non-canonical.

Fix: derive the address width from `CR4.LA57` (the *active* paging mode), not from CPUID capability.

8.1.8 ASID-0 collision

Symptom: the first user thread faulted at its entry point with an instruction abort (`EC=0x21`, "no change in exception level"), despite correct page table entries.

Root cause: the global address-space table started empty, so the first user AS got id 0, which equals `KERNEL_ASID`. `Thread::new` built a kernel thread (EL1) instead of a user thread (EL0), running the user code at EL1 where `PXN=1` forbids execution. The EC decode (`0x21` = fault without EL change) gave the diagnosis: the code was never at EL0.

Fix: pre-populate the table with the kernel AS at id 0, so user spaces always get non-zero ids. A second bug in the same test: `sync_dispatcher` added 4 to `ELR_EL1` on SVC, but AArch64 already sets ELR to the instruction *after* SVC — the +4 skipped the user's loop instruction into zeroed memory.

8.1.9 x86-64 trampoline halted on thread return

Symptom: on x86-64, the async demo's worker completed but the coordinator never resumed.

Root cause: the x86-64 `kernel_thread_trampoline` entered a `hlt` loop when a thread's entry returned, instead of calling `abort()` like the AArch64 trampoline. The CPU halted, freezing the LP.

Fix: make the x86-64 trampoline call `abort` on entry return.

8.2 Key insights

1. **The block-yield-wake cycle is the hard problem.** Designing the completion ABI was straightforward compared to making `wait()` correctly save and restore thread state through the cooperative scheduler. Every scheduling primitive (`sleep`, `wait`, `abort`) exercised a new path and surfaced a new bug. The block-yield-wake path was untested before the async demo.
2. **Booting on hardware finds bugs that static analysis cannot.** The panic-handler recursion, ASID-0 collision, and LA57 paging mismatch were invisible in self-tests and logical reasoning. Every one was found by booting QEMU and observing the actual fault register values.
3. **The observer/waker pattern is the right primitive.** Once `Completion` was made `Observable` and `wait()` used the same path as `sleep()`, the entire completion subsystem fell into place. The exit-observer mechanism is the natural endpoint: a thread exiting *is* the completion event. The worker never references the capability.
4. **The async model genuinely removes the retrofit tax.** On CharlotteOS, `wait(cap)` blocks exactly as `sleep(50ms)` blocks — both register a `Waker` on an `Observable` and yield. There is no emulation, no thread pool, no signal-handler tightrope. The kernel and the runtime agree about what is fundamental.
5. **The sitas traits were essential for co-design.** The `ReactorBackend` trait became the executable specification for the async syscall ABI. The sitas-charlotte backend validates the ABI shapes before any kernel path exists. This bidirectional validation is the core of the collaboration.
6. **The deferred reaper is necessary for any thread that exits.** A thread cannot free its own stack. The reaper pattern (move dead threads to a zombie list, drop them from another context) is the minimum safe discipline.
7. **`RwLock` held-across-call is a systemic hazard.** The Rust pattern of `if let Some(val) = lock().method()` extends the guard's lifetime across the body, which is invisible to the reader.

Bind the value first, then use it. This caused both the sleep heisenbug and the abort deadlock.

9 Glossary

AS (Address Space) A page-table context identified by an `AddressSpaceId` (0 = kernel, >0 = user). Contains the `TTBR0_EL1` (user) and `TTBR1_EL1` (kernel) pointers on AArch64, or `CR3` on x86-64.

Block (verb) To remove a running thread from the scheduler and register its `Waker` on an `Observable`. The thread yields; when the event fires the waker re-admits it.

CQ ring (Completion Queue) A shared-memory ring buffer for zero-syscall completion delivery from the kernel to userspace.

Capability table A per-AS `IdTable<Arc<Completion»` mapping `CompletionCap` (index) to an in-flight or completed operation.

Completion A kernel object representing an in-flight or completed async operation. `Observable`: threads `wait` on it, `complete` signals it.

Exit-observer An `Observer` registered on a `Thread`; fires when the thread is dropped (exits). The ABI's intended completion mechanism: `submit_worker` registers the capability as the worker's exit-observer.

HHDM (Higher Half Direct Mapping) A linear mapping of all physical memory into the kernel's virtual address space, provided by the bootloader (Limine). `PAddr -> *mut T` goes through HHDM.

IdTable A sparse `Vec<Option<T»` with ID recycling. Used for the thread table, address-space table, and capability table.

IPI (Inter-Processor Interrupt) A signal from one LP to another. The kernel's IPI queues are bounded per-LP `ConcurrentQueue` instances carrying `IpiRpc` values.

LP (Logical Processor) One hardware core. The unit of sharding in both the kernel and sitas.

Observable / Observer The event-notification pattern. An `Observable` holds `Weak<dyn Observer>` references; when it fires, it calls `Observer::notify()`.

PerLp<T> A boxed slice of `RwLock<T>`, one per LP. Interrupt-safe (masks interrupts on acquire).

Quantum timer The per-LP periodic timer (10 ms) that drives preemptive context switching in the `RoundRobin` scheduler.

Reaper The deferred-thread-cleanup mechanism: dead threads are moved to `DEAD_THREADS` and dropped from the next thread's context after the context switch.

ShardLocal<T> A lock-free per-LP container using `UnsafeCell<T>` with an owner-check and borrow-flag guard. sitas-derived.

ShardMailbox<M> A typed bounded per-LP queue with `ShardSender<M>` (cloneable) and `ShardReceiver<M>` (single-consumer). First-class backpressure. sitas-derived.

TrapFrame An architecture-specific snapshot of the user register state at the moment an exception was taken. Passed to the syscall handler on AArch64.

Waker An `Observer` wrapping a `ThreadId`. When notified, calls `submit_ready_thread(tid)`.

Yield To voluntarily give up the LP by calling `yield_lp()`. Saves the thread's context and lets the scheduler run the next ready thread.